Snowledge Application Backend

G26

Customer: Juusto Holstein, Arto Hämäläinen
Coach: Tero Ahtee

Meet Your Project Pilots

- Wasay Rizwani — Project Manager
- Nauman Afzal — Product Owner
- Irteza Hammad — Senior Developer
- Waqas Hameed — Senior Developer
- Ibtehaj Abbas — Senior Developer

Let’s Ski Smarter

What Snowledge Is?

Snowledge is a winter-sports app. It started as a simple tool for mountain skiers but has grown into a **nationwide vision across Finland**.
The mission? **Safety first**, with features designed to keep skiers informed, connected, and alive.

What You Can Do With Snowledge

- **Share Snow Conditions:** Skiers can report the current snow in a particular area for everyone else to benefit.
- **Rescue Support:** Users can send a **rescue signal** to others nearby users because apparently, skiing alone is risky.
- **For Ski Resorts:** Manage skier flow and improve visitor satisfaction.
- **For Authorities:** Streamline rescue operations and improve safety.
- **For Partners:** Stay meaningfully connected with skiing enthusiasts.

We Came to Integrate… We Ended Up Rebuilding

So, the grand plan was simple: just integrate Suunto watch into the *existing* system.
Well… turns out that “existing system” was more like an archaeological dig site with lots of mysteries, confusion, and questions.

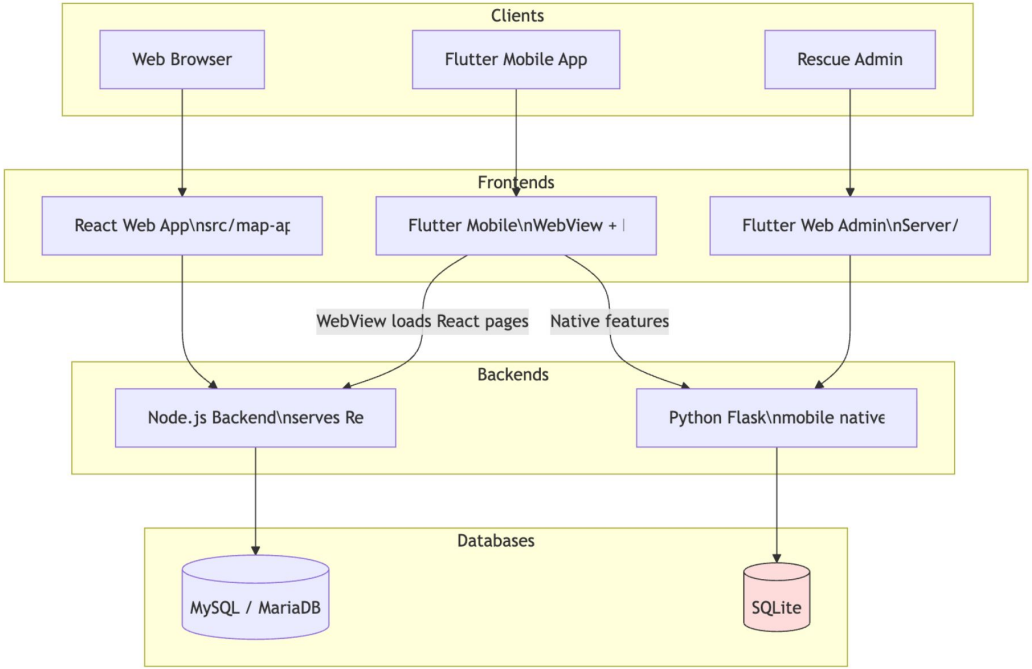
After talking to the customer, we discovered that Group G05 was already rebuilding the whole thing from scratch.

At that point we had two choices:

1. Continue integrating into the old system. (*that was basically scheduled to be thrown out soon*)
2. Team up with G05, join the rebuild party.

We chose the second option (and take charge of the entire backend while they handled the frontend.).
And that’s how our “simple integration task” turned into “let’s rebuild the whole system together.”

The System That Made Us Cry



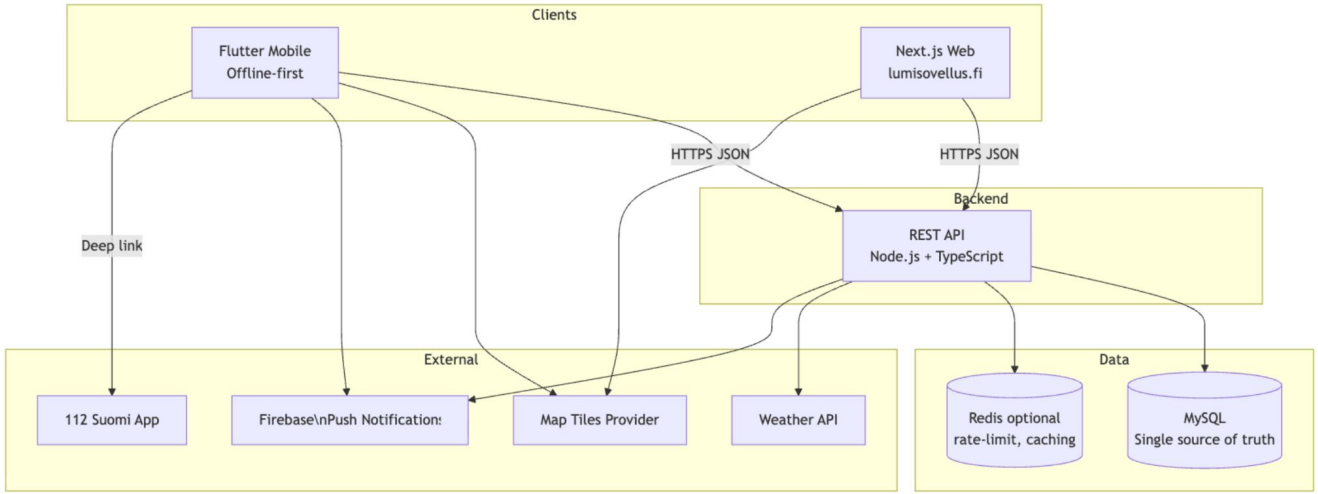
Key Problems

- Mobile app: a WebView loads React pages from Node.js.
- Two databases: MySQL for web content; SQLite for mobile/rescue features. They do not sync.
- This split causes duplicated functionality and inconsistency risks.

Tools and Technologies

Category	Tool / Technology	Rationale / Benefit
Language	Node.js LTS, **TypeScript**	Stable runtime; strong typing prevents bugs and improves code quality.
Core Framework	**Express**	Minimalist, fast, and highly flexible web framework for Node.js.
Database & ORM	**MySQL** via **Prisma**	Battle-tested RDBMS; Prisma provides excellent **Developer Experience (DX)**, safety, and clear schema definition.
API Contract	**REST** with **OpenAPI 3.1**	Standardized, versioned API (`/api/v1`); OpenAPI auto-generation is crucial for documentation and client generation.
Data Integrity	**Zod**	**Strict input validation** on all inputs (body, params, query), enhancing security and reliability.
Client Generation	`openapi-typescript` / `openapi-generator-cli`	**Mandatory automated client generation** for web and mobile, ensuring **type-safe DTOs** and API calls across the entire system.

Freshly Baked & Bug-Free (Hopefully)



Key Improvements

- One backend serves both apps
- No WebView in mobile.
- Offline-first uses local storage on device with background sync and conflict handling.
- All external integrations are called server-side when feasible.

Proof That It Actually Works

What Happened with Suunto Watch Integration?

What Really Happened

- We **didn’t have time** to build the actual watch-to-app integration.
- Why? Because **rebuilding the entire backend** turned into a full-time job.
- So, we focused on **creating a detailed integration plan** for future teams to pick up.
- The plan covers everything from SOS detection on the watch to real-time alerts via mobile and backend.
- Actual development of the watch app and live data flow? That’s on the “to-do” list for the future soldiers.

In short: We laid the groundwork, the *blueprint* but the “fun” coding part is someone else’s mountain to climb.

Thank You!

We’re excited about making skiing safer and smarter